

VIRAJ VAGHASIA

Los Angeles, CA | +1 213-803-5009 | virajvaghiasiamv907@gmail.com | [LinkedIn](#) | [GitHub](#) | [Portfolio](#)

M.S. Computer Science candidate (May 2027) with three years of production software engineering experience, building backend services, data pipelines, and document retrieval systems.

Education

University of Southern California , Los Angeles, CA	Aug 2025 – May 2027
Master of Science in Computer Science	GPA: 3.88/4.00
Dwarkadas J. Sanghvi College of Engineering , Mumbai, India	Aug 2018 – May 2022
Bachelor of Engineering, Information Technology	GPA: 9.38/10
Coursework: Analysis of Algorithms, Information Retrieval, Database Systems, Applied Natural Language Processing	

Technical Skills

Languages: Python, TypeScript, JavaScript, SQL, C++, Bash

Backend & Data: FastAPI, Django, Flask, Node.js, Express, REST APIs, PostgreSQL, MySQL, MongoDB, Redis, SQLite, ETL and batch pipelines

Frontend: React, Next.js, Vue.js, Redux, Tailwind CSS

Infrastructure & Testing: Docker, AWS, GitHub Actions (CI/CD), Git, pytest, Playwright, Modal

Search & Retrieval: Dense embeddings, BM25, hybrid ranking, cross-encoder reranking, evaluation harnesses, Qdrant, vLLM, Ollama

Experience

Research Assistant, USC Information Sciences Institute	May 2026 – Present
<i>Geological literature acquisition and classification pipeline for a critical-minerals research project, Los Angeles, CA</i>	

- Built a **restartable document-acquisition service** in Python checkpointed on a SQLite ledger rather than a run flag, pairing an open-access tier with an authenticated Playwright tier through the USC library proxy; the ledger records **30,096** downloaded articles across **9** journals.
- Replaced per-publisher scrapers with **declarative JSON recipes** covering 4 platforms, plus a generator that proposes extraction rules from one open-access article – deterministic checks decide what ships, not the model.
- Classified crawl failures into **login redirects**, **bot challenges** and **rate-limit blocks** instead of retrying blindly, with escalating **3-to-30-minute backoff** and a halt after three consecutive blocks – an institutional IP block is terminal, not retryable.
- Diagnosed **silent data-loss defects** – discarded supplementary files, an ignored year filter – by comparing files on content hash after filename comparison produced false mismatches, and grew the pytest suite to **348** tests.

Senior Software Engineer, Decimal Point Analytics, Mumbai, India	Jul 2022 – Jul 2025
---	---------------------

- Developed a **real-time financial index platform** in React and FastAPI tracking multiple indices simultaneously, replacing static analyst reports with interactive time-series views.
- Implemented a **maker-checker review workflow** in Django separating the analyst entering data from the reviewer approving it, blocking unreviewed records from publication.
- Led a **Loan Management System** and an **ESG questionnaire portal** across several engineers, onboarding new joiners and gathering requirements from analysts; designed the questionnaire schema so business users could change questions without a code deploy (Vue.js, Next.js, FastAPI).

Projects

SQLAlchemy Upgrade Agent GitHub Python, FastAPI, Qdrant, GitHub Actions, Modal	Jun 2026 – Sep 2026
---	---------------------

- Shipped a **deployed question-answering service** over SQLAlchemy 1.4 → 2.0 migration documentation that cites a source page for every claim and declines when the retrieved pages do not answer.
- Built **hybrid retrieval** over **3,284** passages – dense embeddings with BM25, duplicate collapsing, cross-encoder reranking – raising correct-page-in-top-5 from **0.49 to 0.64** on a **100-question** hand-verified set, measured as paired wins rather than an average.
- Verified answers by **executing each claim against SQLAlchemy 2.0.51** instead of trusting a model's self-report (**47 of 51** correct), and gated retrieval quality in **GitHub Actions** – it blocked a pull request removing the reranker.
- Rejected two proposed retrieval improvements on their own measurements – one dropped recall from 0.64 to 0.58 and was reverted, the other cancelled before implementation once the defect it assumed was shown not to exist.